What This Step Adds

You now have steering (project context), a skill (review workflow), and a power (MCP tools). This final step wraps everything into a dedicated **infra-reviewer** agent - a single specialized role you can invoke anytime that orchestrates all three layers together.

The IDE and CLI use different configuration formats. Pick the path that fits your workflow.

Step 1: Create the Agent Directory

macOS / Linux | Windows (PowerShell)

```
mkdir -p .kiro/agents
```

Step 2: Create the Agent Config

IDE (Subagent) | CLI (Custom Agent)

IDE custom subagents use Markdown files with YAML frontmatter in `.kiro/agents/`.

Create `.kiro/agents/infra-reviewer.md` (Open **terminal** if not exist and run `kiro .kiro/agents/infra-reviewer.md` in it):

```
---
name: infra-reviewer
description: >
  Infrastructure code reviewer for Terraform.
  Reviews for security, cost, compliance, and best practices
  using project context, team standards, and Terraform registry tools.
tools: ["read", "write", "shell"]
model: auto
includePowers: true
---

You are an expert infrastructure engineer specializing in code review.

## How You Work
1. Load the project steering file for project-specific context
   (accounts, regions, naming prefix, tag values)
2. Use the tf-plan-review skill for the structured review workflow,
   team-wide Terraform standards, and compliance scoring
3. Use the power-infra-review power's MCP tools to look up provider docs,
   search the Terraform Registry, and run Checkov scans

## Output Format
Always produce:
- A findings table with severity (Critical/Warning/Info)
- A compliance score
- A PR-ready review comment using the skill's template
- Write the complete review to REVIEW.md in the workspace root
```

> ***includePowers***: *Because `includePowers: true` is set, this agent automatically gets access to the power-infra-review Power's Terraform MCP tools. You don't need to configure MCP servers in the agent file. Compare this with Section 2's `includeMcpJson` which required inline MCP config.*

Step 3: Test the Agent

IDE (Subagent) | CLI (Custom Agent)

Test the complete infra-reviewer with the same sample file:

```
use infra-reviewer subagent to review sample-infra.tf
```

If the agent does not activate automatically, invoke it directly:

```
/infra-reviewer Review sample-infra.tf
```

Verify the output combines all 4 layers:

- **Steering (3.1)**: project context, auto-loaded for .tf files (accounts, naming prefix, tag values)
- **Skill (3.2)**: team-wide Terraform standards, structured review workflow, compliance scoring
- **Power (3.3)**: MCP tools for registry lookups, provider docs, Checkov scanning
- **Agent (3.4)**: specialized role that orchestrates all of the above

Checkpoint
Before proceeding, confirm that the review output includes a findings table with severity levels, a compliance score, and a PR-ready comment. If any layer is missing, check that the steering file, skill, and power are all in place in your workspace.

Step 4: Parallel Subagent Review

Now test parallel subagent invocation. Download these 3 additional Terraform files, each with different infrastructure patterns and issues:

- sample-infra-001.tf - Microservice API (Lambda, API Gateway, DynamoDB)
- sample-infra-002.tf - Data pipeline (ETL workers, RDS, S3 data lake)
- sample-infra-003.tf - Container platform (ECS Fargate, ALB, ECR)

Save all 3 files to your workspace root.

IDE (Subagent) | CLI (Custom Agent)

Ask Kiro to review all 3 files in parallel using the infra-reviewer subagent:

```
invoke infra-reviewer subagent to review sample-infra-00[1-3].tf parallelly and record result in separate REVIEW-00[1-3].md ( no REVIEW.md ) for each subagent.
```

Kiro will spawn multiple subagent instances to review each file concurrently. Each subagent produces its own findings table, compliance score, and PR comment into separate files: REVIEW-001.md, REVIEW-002.md, and REVIEW-003.md.

Note
If Kiro reviews the files sequentially instead of in parallel, that's fine - the results are the same. Parallel execution depends on the model and plan tier.

Verify the results. Each file should produce an independent review:

- `REVIEW-001.md`: API keys in environment variables, public S3 access, overly broad IAM, unauthenticated API Gateway
- `REVIEW-002.md`: Oversized instances (r5.4xlarge), unencrypted EBS/RDS, publicly accessible database, no backups, wide-open security group
- `REVIEW-003.md`: Secrets in container environment, HTTP-only ALB (no HTTPS), no WAF, no autoscaling, ECR scan disabled, single-AZ ECS

Section 3 Summary

Pattern	Section 2 (EOL)	Section 3 (Terraform)
Steering mode	`always`	`fileMatch`
Steering content	Rules + report format + template refs	Project context (accounts, regions, tags)
Skill structure	`SKILL.md` + `references/`	`SKILL.md` + `scripts/` + `references/` + `assets/`
Power	Not built (consumed in Lab 6)	Built from scratch
Agent: MCP access	`includeMcpJson` (inline)	`includePowers: true` (IDE) / `mcpServers` (CLI)
Agent: subagents	Single invocation	Parallel subagent invocation

The Full Stack
You now have a complete infra-reviewer: Steering (project context) + Skill (team standards + review process) + Power (MCP tools) + Agent (dedicated role). Each layer is independent and reusable. The steering file works in any chat, the skill can be invoked standalone, and the power activates for any Terraform conversation.

Ref: IDE Custom Subagents | CLI Custom Agents | CLI Configuration Reference